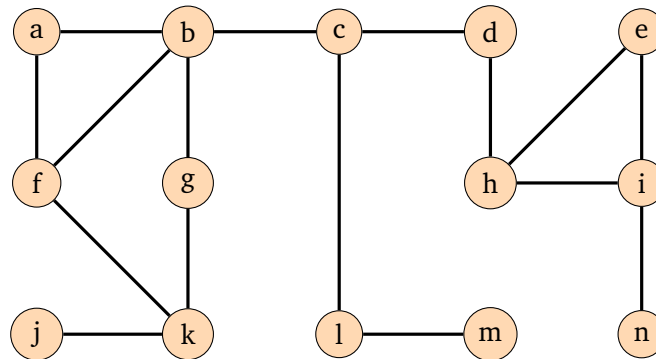


Algorithmen und Datenstrukturen

Aufgabe 1: Spannbaum

Gegeben ist folgender Graph. Konstruiere einen Spannbaum dieses Graphen. Was kannst du über seine Kantenanzahl im Vergleich zu anderen Spannäumen sagen?

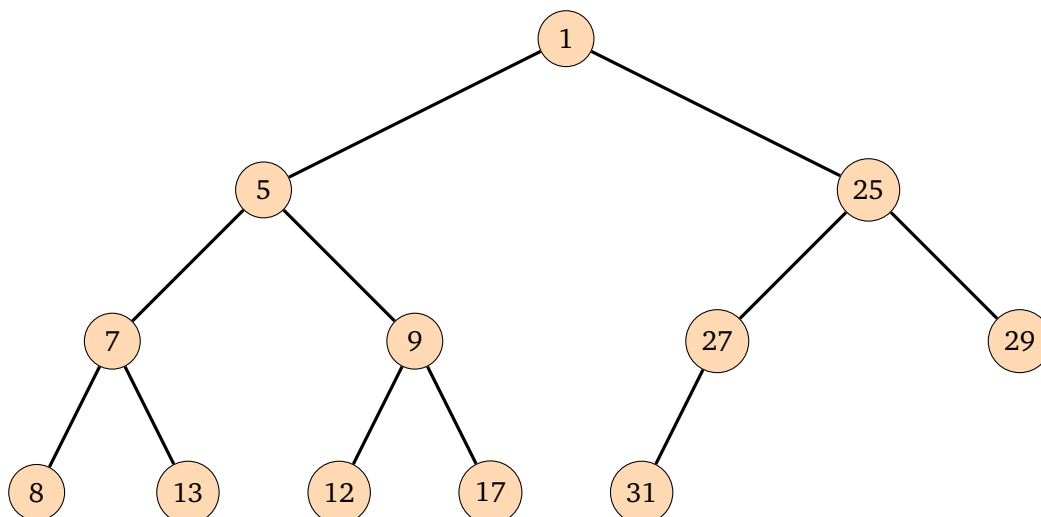


Aufgabe 2: Allgemeines

- Welche Knoten eines ungerichteten Baumes können als Wurzelknoten eines gerichteten Baumes verwendet werden?
- Wieviele verschiedene gerichtete Bäume kann man aus einem ungerichteten Baum machen, wenn die Wurzel gegeben ist?

Aufgabe 3: Binär-Heap

- Gegeben ist folgender Graph. Er ist ein Binär-Heap. Sortiere den Heap so in ein Array ein, dass für jeden Knoten an der Position i dessen Vorgänger an der Position $\lfloor (i-1)/2 \rfloor$ und dessen Nachfolger an den Positionen $2i+1$ bzw. $2i+2$ stehen.



(b–d) siehe Jupyter-Notebook)

Gegeben sind folgende Python-Funktionen, die Operationen auf Binär-Heaps implementieren.

b) Vollziehe die Funktion `decreaseKey` nach, wobei `minHeap` der Graph aus 3a) (siehe PDF-Datei), `index = 9` und `value = -3` ist.

c) Welchen Teil des Aufbaus eines Binär-Heaps führt die untenstehende Funktion `heapify` aus?

d) Implementiere die aus der Vorlesung bekannten Funktionen `insert`, `delete`, `getMin` und `extractMin` unter Zuhilfenahme der gegebenen Funktionen.

Hinweis: es ist stets möglich, dies mit 3 oder weniger Codezeilen zu bewerkstelligen.

```
def decreaseKey(minHeap, index, newValue):
    if minHeap[index] >= newValue:
        minHeap[index] = newValue
        child = index
        heapCondition = False
        while not heapCondition and child > 0:
            parent = int((child-1)/2)
            if minHeap[child] < minHeap[parent]:
                minHeap[parent], minHeap[child] = minHeap[child], minHeap[parent]
                child = parent
            else:
                heapCondition = True

def heapify(minHeap, index):
    lastInnerNode = int((len(minHeap)-1)/2)
    parent = index
    heapCondition = False
    while parent < lastInnerNode and not heapCondition:
        child1 = parent*2+1
        child2 = parent*2+2
        heapCondition = True
        newParent = parent
        if minHeap[child1] < minHeap[newParent]:
            newParent = child1
            heapCondition = False
        if minHeap[child2] < minHeap[newParent]:
            newParent = child2
            heapCondition = False
        minHeap[parent], minHeap[newParent] = minHeap[newParent], minHeap[parent]
        parent = newParent

def transformToHeap(minHeap):
    lastInnerNode = int((len(minHeap)-1)/2)
    for index in range(lastInnerNode, -1, -1):
        heapify(minHeap, index)
```

```

def insert(minHeap, newValue):
    pass

def delete(minHeap, index):
    pass

def getMin(minHeap):
    pass

def extractMin(minHeap):
    pass

minHeap = list(range(10,-1,-1))

print(minHeap)
transformToHeap(minHeap)
print(minHeap)
decreaseKey(minHeap,10,-1)
print(minHeap)
insert(minHeap,3)
print(minHeap)
delete(minHeap, 6)
print(minHeap)

[10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
[0, 1, 4, 2, 6, 5, 8, 3, 7, 9, 10]
[-1, 0, 4, 2, 1, 5, 8, 3, 7, 9, 6]
[-1, 0, 3, 2, 1, 4, 8, 3, 7, 9, 6, 5]
[-1, 0, 3, 2, 1, 4, 5, 3, 7, 9, 6]

```